

Part 9: Path Problems

Root-to-leaf, any-to-any, and backtracking

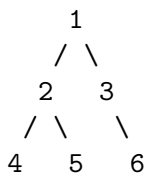
Three kinds of “path”

Problem statements use the word “path” loosely, and the three meanings need completely different code. Sort this out before you write a line.

Kind	Shape	Typical problems	Technique
Root to leaf	starts at the root, ends at a leaf	112, 113, 129, 257	preorder, carry state down
Downward path	starts anywhere, goes only down	437 Path Sum III	preorder plus prefix sums
Any to any	goes up and then down, turns once	124 Max Path Sum, 543 Diameter	postorder, bottom-up

The signature question: **can the path turn a corner?** If yes, it is a bottom-up problem and you need the “record versus return” split from Parts 6 and 7. If no, it is a top-down problem and you carry information down as a parameter.

Working tree:



Its root to leaf paths are 1-2-4, 1-2-5, and 1-3-6.

Root to leaf: the basic shape

Information flows **down**. Each child needs to know what happened above it. So you pass an extra parameter, and the recursion is preorder.

```
void dfs(TreeNode* root, int sumSoFar) {
    if (root == NULL) return;
```

```

sumSoFar += root->val;

if (root->left == NULL && root->right == NULL) {
    // this is a leaf, sumSoFar is a complete root-to-leaf sum
    return;
}

dfs(root->left, sumSoFar);
dfs(root->right, sumSoFar);
}

```

Two things to be careful about:

- The leaf test is `left == NULL && right == NULL`. Both. Always.
- `sumSoFar` is passed **by value**, so each branch automatically gets its own copy. There is nothing to undo. When you pass by value, the “backtracking” happens for free.

Path Sum (LeetCode 112)

Does some root to leaf path add up to target?

```

bool hasPathSum(TreeNode* root, int target) {
    if (root == NULL) return false;

    target -= root->val;

    if (root->left == NULL && root->right == NULL)
        return target == 0;

    return hasPathSum(root->left, target) || hasPathSum(root->right, target);
}

```

Instead of accumulating a sum, we subtract as we descend, and ask whether the remaining target is exactly 0 at a leaf. Same thing, one variable fewer.

Trace on our tree with target 7: $7 - 1 = 6$ at node 1; $6 - 2 = 4$ at node 2; $4 - 4 = 0$ at leaf 4, so true.

Watch the base case. `if (root == NULL) return target == 0;` is **wrong**. Consider a root with value 5, target 5, and only a left child. Node 5’s right child is NULL, and NULL would report success even though we never reached a leaf. The leaf test must be done at the leaf, not at NULL.

Time $O(N)$, space $O(H)$.

Maximum root to leaf sum

```

int maxRootToLeaf(TreeNode* root) {
    if (root == NULL) return INT_MIN;           // no path exists here
    if (root->left == NULL && root->right == NULL) return root->val;

    return root->val + max(maxRootToLeaf(root->left), maxRootToLeaf(root->right));
}

```

This one is written bottom-up instead, and it is worth comparing with the previous two. Both directions work for root to leaf problems. Choose the one that makes the code shorter. Here `INT_MIN` for NULL means “this branch offers nothing”, so it always loses the max. Careful: if the tree can have

very negative values, `root->val + INT_MIN` overflows. Guard the single child case explicitly, or use `long long`.

Collecting the actual paths: backtracking

LeetCode 257 asks for all root to leaf paths, printed. Now we need the path itself, not just a number. Storing a `vector<int>` by value at every call would be $O(N)$ copying per node. Instead keep **one** shared vector and undo your change on the way out.

```
void dfs(TreeNode* root, vector<int>& path, vector<vector<int>>& res) {
    if (root == NULL) return;

    path.push_back(root->val);           // do

    if (root->left == NULL && root->right == NULL) {
        res.push_back(path);           // record a copy
    } else {
        dfs(root->left, path, res);
        dfs(root->right, path, res);
    }

    path.pop_back();                   // undo
}
```

The push_back and pop_back must pair up. That pairing is what backtracking means: the shared state is exactly correct when you enter a node and exactly restored when you leave it.

Dry run on our tree

```
enter 1  path=[1]
  enter 2  path=[1,2]
    enter 4  path=[1,2,4]  leaf -> record [1,2,4]
    leave 4  path=[1,2]
    enter 5  path=[1,2,5]  leaf -> record [1,2,5]
    leave 5  path=[1,2]
  leave 2  path=[1]
  enter 3  path=[1,3]
    left is NULL, returns immediately
    enter 6  path=[1,3,6]  leaf -> record [1,3,6]
    leave 6  path=[1,3]
  leave 3  path=[1]
leave 1  path=[]
```

Result: `[[1,2,4],[1,2,5],[1,3,6]]`.

Delete the `pop_back` and the second path comes out as `[1,2,4,5]`. That is the classic symptom: paths that grow and never shrink.

Time $O(N * H)$ because copying each recorded path costs $O(H)$ and there can be $O(N)$ leaves. **Space $O(H)$** for the path plus the output.

Path Sum II (LeetCode 113)

Combine the two ideas above: carry the remaining target down, and keep the path with push and pop. Record path when you are at a leaf and the remaining target is 0. Write it yourself before looking at any editorial. It is the single best exercise in this part.

Sum Root to Leaf Numbers (LeetCode 129)

Each root to leaf path spells a number, for example 1-2-4 means 124. Carry the number down:

```
int dfs(TreeNode* root, int cur) {
    if (root == NULL) return 0;

    cur = cur * 10 + root->val;

    if (root->left == NULL && root->right == NULL) return cur;

    return dfs(root->left, cur) + dfs(root->right, cur);
}
// call dfs(root, 0)
```

Here NULL returning 0 is safe, because a missing child simply contributes nothing to the sum.

Path from the root to a specific node

Useful on its own and as one way to solve LCA.

```
bool findPath(TreeNode* root, int target, vector<int>& path) {
    if (root == NULL) return false;

    path.push_back(root->val);

    if (root->val == target) return true;           // found, keep the path

    if (findPath(root->left, target, path)) return true;
    if (findPath(root->right, target, path)) return true;

    path.pop_back();                               // not on this branch, undo
    return false;
}
```

Note the asymmetry: when we succeed we deliberately do **not** pop, because the caller wants the path preserved. Every return true skips the undo. That is intentional, and it is worth staring at for a moment.

Any to any: maximum path sum

LeetCode 124. The path may start and end anywhere, and it turns at most once. This is the diameter skeleton with sums.

```

class Solution {
    int best;

    int gain(TreeNode* root) {
        if (root == NULL) return 0;

        int L = max(0, gain(root->left));    // a negative branch is worse than nothing
        int R = max(0, gain(root->right));

        best = max(best, root->val + L + R);  // path turning HERE, uses both sides

        return root->val + max(L, R);        // for my parent, one side only
    }

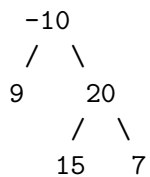
public:
    int maxPathSum(TreeNode* root) {
        best = INT_MIN;
        gain(root);
        return best;
    }
};

```

Three details, all of which are commonly missed:

1. **max(0, ...)**: if a subtree's best downward sum is negative, do not attach it. Take nothing instead. This is only legal because a path is allowed to stop at the current node.
2. **best starts at INT_MIN**, not 0. If every value is negative, the answer is the single largest (least negative) node, and starting at 0 would wrongly return 0.
3. **Record val + L + R, return val + max(L, R)**. Exactly the diameter split. Returning val + L + R upward would let the parent extend a path that already turned, which is not a path.

Dry run



```

gain(9)   = 9,    best = max(INT_MIN, 9)           = 9
gain(15)  = 15,   best = max(9, 15)                = 15
gain(7)   = 7,    best = max(15, 7)                = 15
gain(20): L=15 R=7  best = max(15, 20+15+7)        = 42   return 20 + 15 = 35
gain(-10): L=9 R=35 best = max(42, -10+9+35)        = 42   return -10 + 35 = 25

```

answer = 42 (the path 15 -> 20 -> 7)

Downward path starting anywhere

LeetCode 437, Path Sum III: count paths that go downward but need not start at the root or end at a leaf.

The beginner solution: for every node, run a “count downward paths starting here” DFS. That is $O(N^2)$, and it passes on LeetCode but is not the intended answer.

The good solution reuses a classic array trick. Keep the running sum from the root, and a hash map from prefix sum to how many times it has occurred on the current root-to-node path. At a node with running sum S , the number of valid paths ending here is $\text{count}[S - \text{target}]$. Add your prefix sum to the map on the way in, remove it on the way out. That undo is the same backtracking discipline as above. **$O(N)$ time, $O(H)$ space.**

You do not need to master this on day one. Recognise it as “prefix sum on a tree path” and come back to it after you are comfortable with the rest.

Common mistakes

- **Treating a node with one child as a leaf.** Both children must be NULL.
- **Doing the target check at NULL instead of at a leaf.**
- **Forgetting pop_back.** Paths keep growing.
- **Popping after a successful find in findPath.** That destroys the answer you just built.
- **Passing the path by value everywhere.** It is correct but copies $O(H)$ at every node.
- **Starting best at 0 in maximum path sum.** Fails on all-negative trees.
- **Returning val + L + R upward.** The parent cannot use a path that already turned.
- **Forgetting max(0, ...)** and dragging negative subtrees into the answer.
- **Overflow.** Sums of 10^4 nodes with values up to 10^4 fit in `int`, but when in doubt use `long`.

Check yourself

1. Which three kinds of path are there, and what single question separates them?
2. Why does passing `sumSoFar` by value need no explicit backtracking?
3. Why is `if (root == NULL) return target == 0;` wrong in Path Sum?
4. In `findPath`, why is there no `pop_back` on the successful return?
5. In maximum path sum, why is `max(0, gain(child))` used, and what would break without it?
6. Why must `best` start at `INT_MIN` and not 0 in problem 124?
7. What is the time complexity of collecting all root to leaf paths, and where does the extra factor come from?